

PeopleCRM PostgreSQL Schema Review

A full assessment of schema design, indexing, performance posture, and security for the PeopleCRM multi-tenant database — with prioritized, executable remediation SQL.

DATABASE	pplcrm — PostgreSQL 14.18 (Homebrew, local development instance)
SCOPE	64 tables · 245 indexes · 160 foreign keys · 24 MB (development data)
DATE	July 6, 2026
PREPARED FOR	Zee — PeopleCRM engineering
METHOD	Read-only catalog & statistics inspection, cross-referenced against the application code (Kysely repositories, migrations, auth & worker modules). No database changes were made.

Contents

1. Executive summary
2. Current schema overview — domains, diagrams, and table inventory
3. Design findings and recommendations
4. Indexing findings and recommendations
5. Performance findings and recommendations
6. Security findings and recommendations
7. Prioritized action plan

Method & limitations

All observations come from read-only queries against the live development database (catalog tables, `pg_stat_user_tables`, `pg_stat_user_indexes`, `pg_stats`, role/ACL views) and from reading the application source. Findings that depend on the code cite the file. Limitations to keep in mind when weighing the evidence:

- **`pg_stat_statements` is not installed**, so there is no per-query timing data. Query-shape analysis is based on the Kysely repository code instead (which is authoritative for *what* runs, if not *how often*).
- **Usage counters reflect the development workload** (largely the test suite). An index with zero scans here may still be essential in production, and the heavy sequential-scan counts on tiny tables are the planner behaving correctly, not a defect. Structural findings (duplicates, missing constraints) are workload-independent and stand on their own.
- **`pgstattuple` is not installed**; bloat was assessed via `n_dead_tup` and relation sizes — adequate at 24 MB.
- **Server settings are dev-machine defaults** (`ssl=off`, `trust` auth, 128 MB `shared_buffers`). These are judged in that context, with a separate production checklist rather than alarm bells.
- Backup/DR tooling was not in scope on this local instance; §6 lists what production must have.

This review complements the application-level `SECURITY-REVIEW.md` already in the repository. Items that review already tracks (session revocation, token hashing, rate limiting) are acknowledged and cross-referenced, not re-litigated.

1 Executive summary

The PeopleCRM schema is in fundamentally good shape for a product at this stage. The things that are hardest to retrofit are already right: nearly every table carries a `tenant_id`, the newer tables use a composite `(id, tenant_id)` key pattern that makes cross-tenant references structurally impossible, credentials are hashed with modern algorithms (argon2id for passwords, SHA-256 for high-entropy tokens), heavy work goes through a transactional outbox with `SKIP LOCKED` workers, and text search is backed by a deliberate trigram/tsvector index strategy.

The problems found are mostly the sediment of fast iteration: constraints and indexes added twice under different names, older tables that predate the conventions the newer tables follow, and a handful of dev-instance settings that must not reach production. Almost everything in the top ten is fixable with short, low-risk migrations.

Top findings

SEV	REF	FINDING	IMPACT → EFFORT
P1	D-1	11 duplicate foreign-key constraints, several with conflicting ON DELETE actions (e.g. <code>profiles.auth_id</code> has both a NO ACTION and a CASCADE constraint). Every child write pays double referential-integrity checks, and the effective delete behavior is not what the older constraint's author intended.	Correctness & write cost → Low (drop 11 constraints)
P1	S-1	Tenant isolation exists only in the application layer. No row-level security; the ESLint rule (<code>no-unscooped-db-query</code>) is the sole guard, and it self-documents shapes it cannot catch. For a product holding political-campaign PII, RLS is the right second wall.	Security depth → High (phased)
P1	P-1	PostgreSQL 14 reaches end-of-life November 12, 2026 — four months away. Plan the jump to 16/17 before launch while the database is still 24 MB and the migration is trivial.	Support & perf → Medium
P1	S-4	Microsoft/Google OAuth access and refresh tokens are stored in plaintext (<code>ms_oauth_tokens</code> , <code>google_oauth_tokens</code>). These grant live mailbox access; they are the only secrets in the database not hashed or encrypted.	Security → Medium
P2	I-1	15 exactly-duplicate indexes (same table, same key columns) — pure write amplification and wasted cache. Exact DROP INDEX list provided.	Write perf → Low
P2	I-3	Join/map tables have no index on their second foreign key , so deleting a parent row (a tag, a list, a person) makes referential-integrity triggers sequentially scan every map table. Invisible at dev size; O(table) per deleted parent row in production.	Delete perf at scale → Low
P2	D-5	Nine tables use timestamp without time zone (the whole email domain, donations, files) while the rest of the schema — including other columns of the same <code>emails</code> table — uses <code>timestampz</code> . Correct only for as long as every process shares one TZ setting.	Latent correctness → Medium
P2	D-2	profiles has no primary key (only a nullable-pattern unique index), plus a redundant conflicting FK on <code>auth_id</code> . One-line fix.	Integrity → Low
P2	D-4	*_oauth_tokens.tenant_id is text while every other table uses <code>bigint</code> — no FK to tenants is possible, joins require casts, and the tenant-safety pattern skips these highly sensitive tables.	Integrity & security → Medium
P2	S-2	The application connects as the table owner — it can DROP TABLE. Production needs an unprivileged runtime role, and PG14's default CREATE-on-public for all users should be revoked.	Least privilege → Low/Med

What is already good

- **Tenant scoping by construction** on newer tables: composite PKs (`id, tenant_id`) and composite FKs such as `event_registrations → persons(id, tenant_id)` make cross-tenant rows unrepresentable.
- **Credential hygiene**: argon2id (64 MB, t=3) for passwords; session IDs, refresh tokens, reset codes, 2FA codes, Zapier keys and donation webhook tokens all stored as SHA-256 digests (per the current branch and recent commits).
- **Job architecture**: transactional outbox (`background_jobs` inserted in the same transaction as business writes), `FOR UPDATE SKIP LOCKED` claiming, LISTEN/NOTIFY wake-ups, bounded worker concurrency.
- **Search strategy**: deliberate GIN trigram indexes on exactly the columns the grid searches with `ILIKE '%...%'`, plus tsvector columns for full-text.
- **SQL injection posture**: all queries parameterized through Kysely; the one `sql.raw()` use interpolates only server-side allow-listed column mappings, never client strings (verified in `base.repo.ts`).
- **CHECK-constraint enums** (text + CHECK) on newer status columns — the right pattern in Postgres; §3 lists the older columns that still need one.

HOW TO READ THIS REPORT

Findings are labeled **P0** (exploitable/broken now) → **P3** (polish), matching the severity legend of the repository's `SECURITY-REVIEW.md`. Every finding carries executable SQL and a risk note. Section 7 sequences them into quick wins and larger refactors. Since there are no production users yet, several fixes that would normally need `CONCURRENTLY / NOT VALID` choreography can simply be run as ordinary migrations now — each finding states both paths.

2 Current schema overview

The database is a classic shared-schema multi-tenant design: one `tenants` table, and a `tenant_id` `bigint` column on essentially every business table. Two identifier styles coexist deliberately — `bigserial` surrogate keys for internal entities, and `uuid` keys for objects whose IDs travel to the public internet (`web_forms`, `form_submissions`, OAuth token rows). Six functional domains emerge; each is diagrammed below with its key relationships (audit columns `createdby_id` / `updatedby_id` → `authusers` exist on most tables and are omitted from the diagrams for legibility).

METRIC	VALUE	METRIC	VALUE
Tables (public schema)	64	Foreign keys	160 (11 are duplicates — §3)
Indexes	245 (15 exact duplicates — §4)	CHECK constraints	22
Primary keys	63 of 64 (profiles missing — §3)	Unique constraints	36
Extensions	pg_trgm (fuzzy search), pgcrypto (UUIDs; encryption unused), plpgsql		
Triggers	26 × <code>set_updated_at()</code> , LISTEN/NOTIFY on <code>background_jobs</code> and <code>webhook_events</code> ; no audit triggers		

2.1 Tenancy & identity

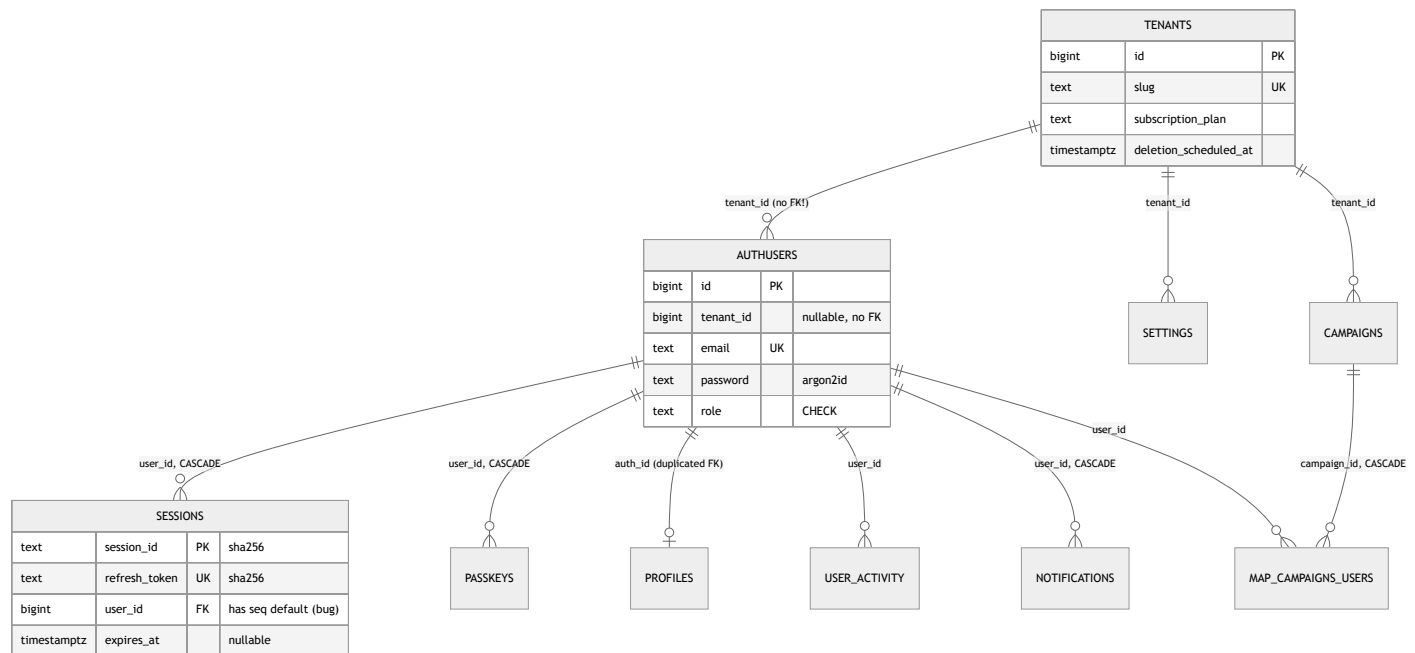


Fig. 1 — Tenancy & identity. `authusers.tenant_id` and `passkeys.tenant_id` carry no FK; `profiles` has no PK.

2.2 People & households

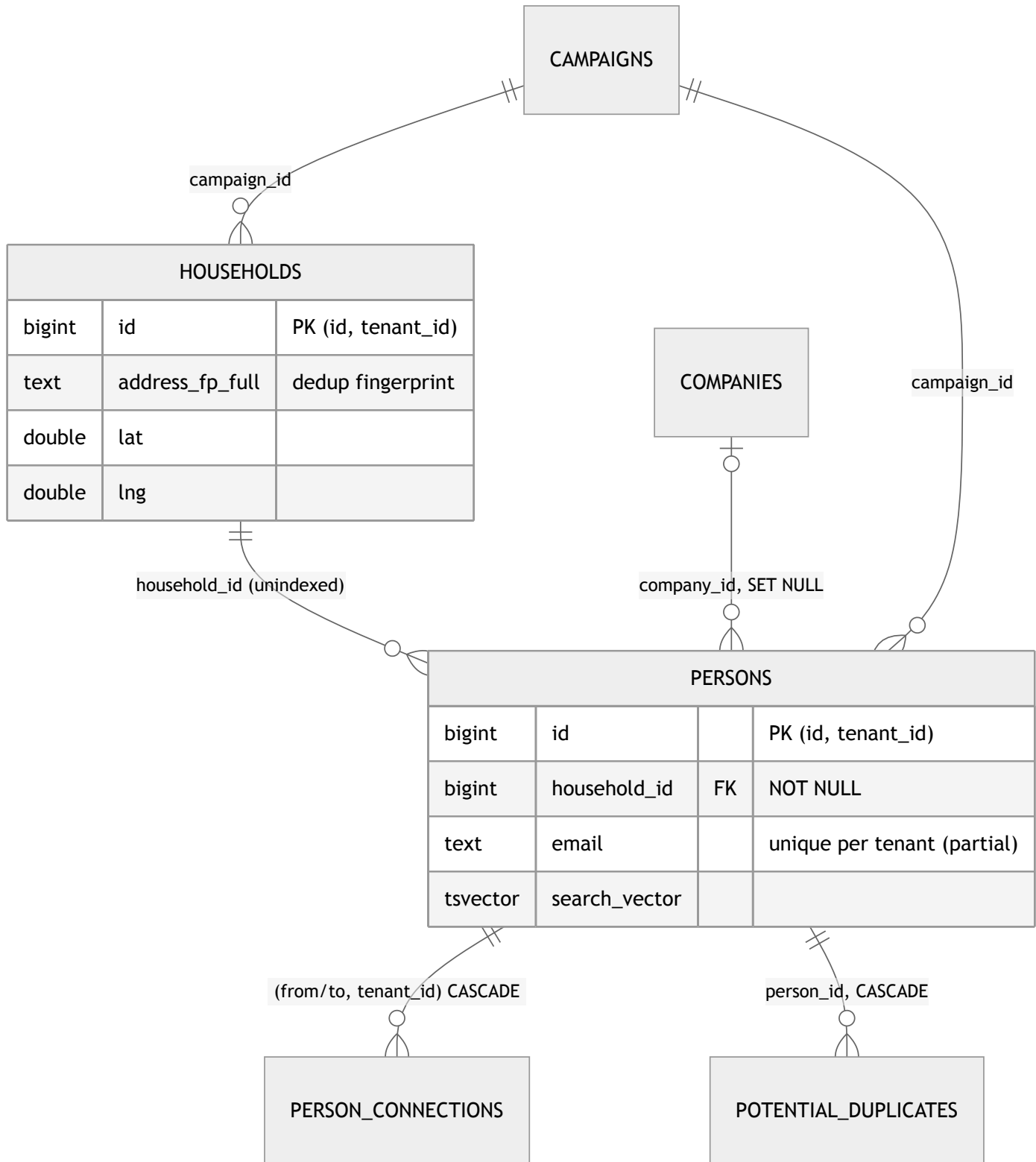


Fig. 2a — People & households core. `persons.household_id` is NOT NULL (placeholder-household pattern) and unindexed.

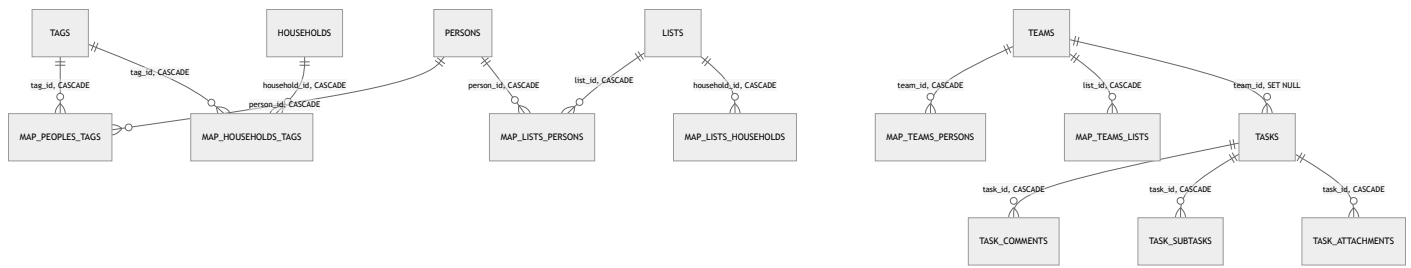


Fig. 2b — Organizing: tags, lists, teams, tasks. All map tables have PK (tenant_id, x_id, y_id) ; their second FK columns are unindexed (§4, l-3).

2.3 Email workspace

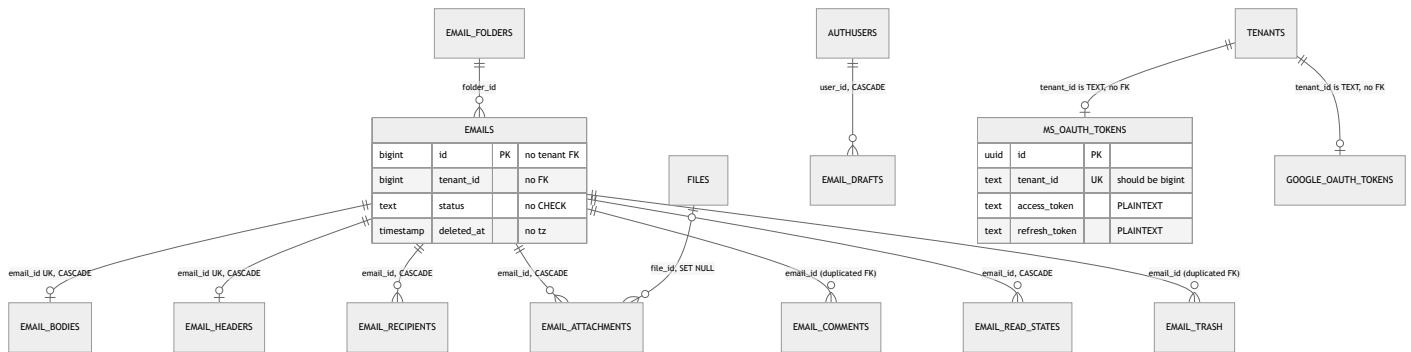


Fig. 3 — Email workspace. The entire domain uses timestamp without time zone ; no table in it has an FK to tenants .

2.4 Outreach — newsletters, workflows, web forms

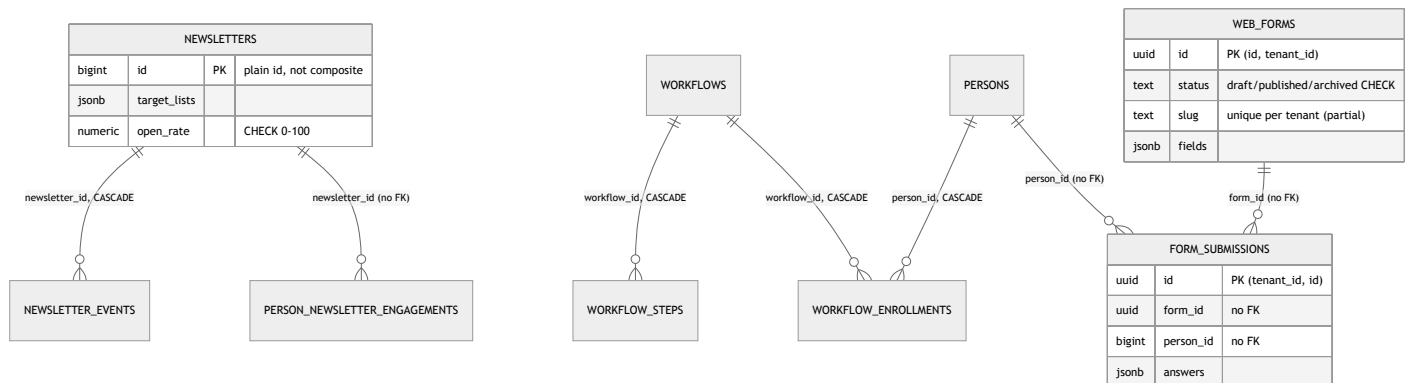


Fig. 4 — Outreach. form_submissions and person_newsletter_engagements have no foreign keys at all.

2.5 Events & volunteers

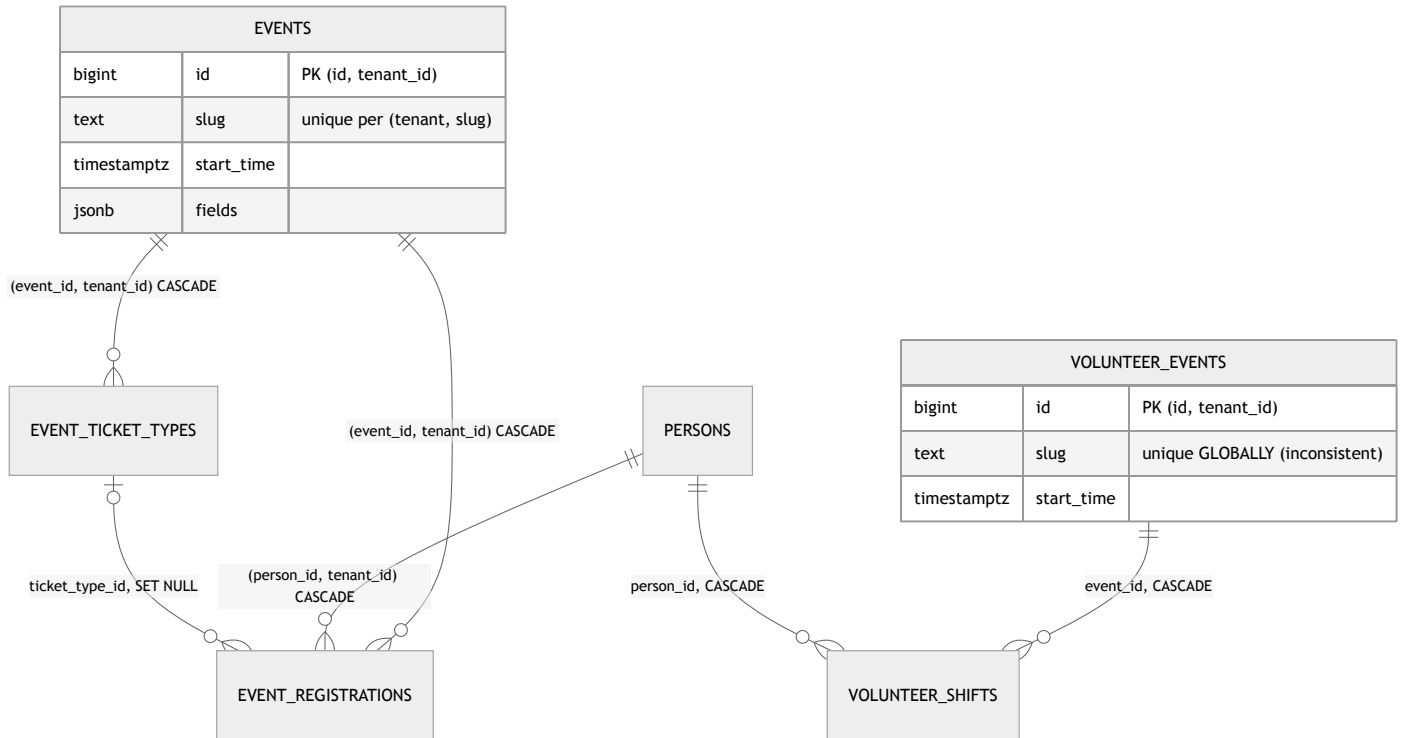


Fig. 5 — Two parallel event stacks. `events` children use exemplary composite tenant-safe FKs; `volunteer_*` predates that pattern.

2.6 Money, integrations & operations

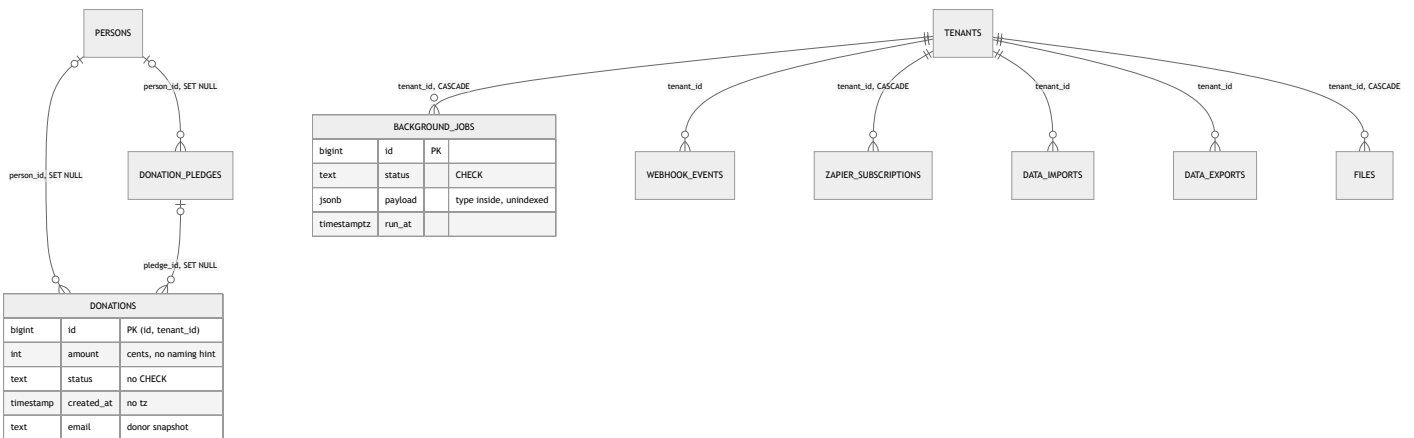


Fig. 6 — Money & ops. `donation_periods` (not shown) has `tenant_id` with no FK; `webhook_events` mirrors the jobs pattern for Stripe.

2.7 Table inventory

Row counts are development data and indicate shape, not scale. “PK shape” distinguishes the tenant-composite pattern from plain surrogate keys. *X* marks a missing tenant FK on a table that has a `tenant_id` column.

DOMAIN	TABLE	ROWS	PK SHAPE	TENANT FK	NOTES

Tenancy & identity	tenants	53	id	—	Billing/Stripe columns, slug (partial unique), lifecycle timestamps
	authusers	68	id	✗ none	Credentials; role CHECK; 2FA columns; tenant_id nullable
	profiles	30	none	✓	No PK; duplicate FK on auth_id; user contact details
	sessions	42	session_id (text)	✗ none	Hashed tokens; user_id has stray sequence default; 3 duplicate indexes
	passkeys	1	id (identity)	✗ none	WebAuthn credentials; duplicate credential_id index
	settings	98	id	✓ CASCADE	Per-tenant KV (jsonb); unique (tenant_id, key)
	user_activity	44	(id, tenant_id)	✓	App-level activity log; 2 duplicate index pairs; entity_id is text
	notifications	6	(id, tenant_id)	✓	Per-user notifications; read flag
	campaigns + map_campaigns_users	10	(id, tenant_id)	✓	Campaign scoping for persons/households
People & organizing	persons	30	(id, tenant_id)	✓	17 indexes (6 GIN); partial unique lower(email); opt-in columns
	households	30	(id, tenant_id)	✓ (name typo)	14 indexes; geocoding + address fingerprints; placeholder pattern
	companies	1	(id, tenant_id)	✓	tsvector + 3 trgm indexes
	tags + 2 map tables	103	(id, tenant_id)	✓	Unique (tenant, name, type); map PKs tenant-leading
	lists + 2 map tables	10	(id, tenant_id)	✓	Dynamic lists (jsonb definition, status CHECK)
	teams + 2 map tables	10	(id, tenant_id)	✓	captain (person) + lead (authuser) FKs
	tasks + 3 child tables	30	(id, tenant_id)	✓	status CHECK; priority/subtask status lack CHECKs
	person_connections, potential_duplicates	~0	(id, tenant_id) / id	✓	Graph edges w/ composite FKs; dedup queue w/ partial uniques

Email workspace	emails	19	id	✗	Mixed tz in same table; status no CHECK; body duplicated with email_bodies
	email_bodies, email_headers	19	id	✗	1:1 splits (unique email_id); duplicate email_id indexes
	email_recipients, email_attachments, email_comments, email_read_states, email_trash, email_drafts, email_folders	~90	mixed	✗ mostly	email_comments FK duplicated + unindexed; folders per tenant, no FK
	ms_oauth_tokens, google_oauth_tokens	1	uuid	✗ (text col)	Plaintext tokens; tenant_id text; duplicate tenant index
	files	~5	id	✓ CASCADE	Azure blob metadata; sha256 partial index; timestamps no tz
Outreach	newsletters	10	id	✓	Aggregate engagement counters with range CHECKs
	newsletter_events	0	id	✓	SendGrid webhook events; ip/user_agent PII; global unique sg_event_id
	person_newsletter_engagements	0	(tenant, nl, email)	✗ no FKs at all	Per-recipient rollup keyed by email
	workflows + steps/enrollments; web_forms, form_submissions	~80	mixed	✓ / ✗	form_submissions has zero FKs; enrollments missing unique (workflow, person)
Events & volunteers	events + ticket_types + registrations	~1	(id, tenant_id)	✓ (via composite)	Model pattern: composite tenant-safe FKs, CHECKs on capacity/price
	volunteer_events + volunteer_shifts	~20	(id, tenant_id)	✓	Global slug uniqueness (vs per-tenant on events); shift status CHECK
Money & ops	donations, donation_pledges, donation_periods	~0	(id, tenant_id)	✓ / ✓ / ✗	Donor snapshot columns (intentional); amounts in integer cents; donations tz-less
	background_jobs	2,152	id	✓ CASCADE	Outbox; largest table in dev — completed rows never pruned
	webhook_events, zapier_subscriptions	~1	id	✓	Stripe event dedupe (unique stripe_event_id); Zapier REST hooks
	data_imports, data_exports, kysely_migration(+lock)	~15	(id, tenant) / name	✓ / —	Import/export bookkeeping; migration ledger

3 Design findings and recommendations

P1 D-1 Eleven duplicate foreign-key constraints, several with conflicting ON DELETE actions

Seven tables carry two FK constraints over the same column referencing the same parent — one older `fk_*`-named constraint (mostly `ON DELETE NO ACTION`) and one newer `*_fkey`-named constraint (mostly `ON DELETE CASCADE`). The pattern traces to migrations that added corrected constraints under new names without dropping the originals (the `2026-06-25-fix-onDelete-behavior` migration dropped only the names it knew about).

WHY IT MATTERS

Every insert/update of the child column fires two RI triggers, and every parent delete evaluates both actions. Where the actions differ (`NO ACTION` + `CASCADE`), the delete still cascades — the `CASCADE` trigger removes the child rows before the `NO ACTION` check runs — so the older constraint provides no protection while suggesting to readers that it does. It is dead weight with misleading semantics.

FIX — DROP THE OLDER TWIN, KEEP THE CONSTRAINT WHOSE ON DELETE THE FIX-ONDELETE MIGRATION INTENDED

```
ALTER TABLE email_comments      DROP CONSTRAINT fk_email_comments_email;      -- keep
email_comments_email_id_fkey (CASCADE)
ALTER TABLE email_trash         DROP CONSTRAINT fk_email_trash_email;         -- both CASCADE; keep
email_trash_email_id_fkey
ALTER TABLE map_lists_households DROP CONSTRAINT fk_map_lists_households_household; -- keep
map_lists_households_household_id_fkey (CASCADE)
ALTER TABLE map_lists_households DROP CONSTRAINT fk_map_lists_households_list;  -- keep
map_lists_households_list_id_fkey (CASCADE)
ALTER TABLE map_lists_persons   DROP CONSTRAINT fk_map_lists_persons_person;   -- keep
map_lists_persons_person_id_fkey (CASCADE)
ALTER TABLE map_lists_persons   DROP CONSTRAINT fk_map_lists_persons_list;     -- keep
map_lists_persons_list_id_fkey (CASCADE)
ALTER TABLE map_teams_lists     DROP CONSTRAINT fk_map_teams_lists_list;      -- keep
map_teams_lists_list_id_fkey (CASCADE)
ALTER TABLE map_teams_lists     DROP CONSTRAINT fk_map_teams_lists_team;      -- keep
map_teams_lists_team_id_fkey (CASCADE)
ALTER TABLE map_teams_persons   DROP CONSTRAINT fk_map_teams_persons_person;   -- keep
map_teams_persons_person_id_fkey (CASCADE)
ALTER TABLE map_teams_persons   DROP CONSTRAINT fk_map_teams_persons_team;    -- keep
map_teams_persons_team_id_fkey (CASCADE)
ALTER TABLE profiles           DROP CONSTRAINT fk_profiles_auth_id;           -- keep
profiles_auth_id_fkey (CASCADE)
```

RISK

Each drop takes a brief `ACCESS EXCLUSIVE` lock — instant at any realistic size. Confirm per pair that `CASCADE` is the intended semantics (it matches the documented intent for join/owned rows). Apply as one migration and refresh `schema.sql`.

P2 D-2 `profiles` has no primary key

`profiles` is the only table without a PK — it has `id bigint NOT NULL DEFAULT nextval(...)` and a unique index `profiles_id_key`, but no PK constraint. Kysely doesn't care, but logical replication, many ORMs/tools, and `REPLICA IDENTITY` do; it also signals “unfinished” to every reader.

FIX

```
ALTER TABLE profiles ADD CONSTRAINT profiles_pkey PRIMARY KEY USING INDEX profiles_id_key;
-- Optionally tighten the 1:1 with authusers while here:
ALTER TABLE profiles ALTER COLUMN auth_id SET NOT NULL; -- verify no NULLs first
ALTER TABLE profiles ADD CONSTRAINT profiles_auth_id_unique UNIQUE (auth_id);
```

RISK

None for the PK promotion (reuses the existing index; the index is renamed to the constraint name). The NOT NULL/UNIQUE tightening requires checking existing rows (`SELECT count(*) FROM profiles WHERE auth_id IS NULL`) and confirming profiles are truly 1:1 with users.

P2 D-3 Missing foreign keys on tenant-scoped and reference columns

FK coverage is inconsistent by domain age. Tables holding a plain `tenant_id bigint` with *no* FK to `tenants`: `authusers`, `sessions`, `passkeys`, `donation_periods`, and the whole email domain (`emails`, `email_folders`, `email_bodies`, `email_headers`, `email_recipients`, `email_attachments`, `email_comments`). Worse, `form_submissions` and `person_newsletter_engagements` have **no foreign keys at all** — a deleted web form or person leaves orphaned submissions silently. `tags.createdby_id/updatedby_id` also lack the audit FKs every sibling table has.

WHY IT MATTERS

Without FKs, tenant deletion (the app supports scheduled tenant deletion) and record merges must remember every orphan by hand; one missed table = dangling PII that no tenant-scoped query will ever show again but that still sits in the database and in exports/backups.

FIX (REPRESENTATIVE; SAME PATTERN FOR EACH LISTED TABLE)

```
-- Tenant links (choose CASCADE for owned rows, matching settings/files precedent):
ALTER TABLE emails          ADD CONSTRAINT fk_emails_tenant          FOREIGN KEY (tenant_id)
REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE email_folders   ADD CONSTRAINT fk_email_folders_tenant   FOREIGN KEY (tenant_id)
REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE authusers       ADD CONSTRAINT fk_authusers_tenant       FOREIGN KEY (tenant_id)
REFERENCES tenants(id);      -- NO ACTION: users outlive nothing silently
ALTER TABLE passkeys        ADD CONSTRAINT fk_passkeys_tenant        FOREIGN KEY (tenant_id)
REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE donation_periods ADD CONSTRAINT fk_donation_periods_tenant FOREIGN KEY (tenant_id)
REFERENCES tenants(id);

-- form_submissions: use the composite pattern so a submission can never point across tenants:
ALTER TABLE form_submissions
  ADD CONSTRAINT fk_form_submissions_form  FOREIGN KEY (form_id, tenant_id) REFERENCES web_forms(id,
tenant_id) ON DELETE CASCADE,
  ADD CONSTRAINT fk_form_submissions_person FOREIGN KEY (person_id, tenant_id) REFERENCES persons(id,
tenant_id) ON DELETE CASCADE;

-- person_newsletter_engagements:
ALTER TABLE person_newsletter_engagements
  ADD CONSTRAINT fk_pne_newsletter FOREIGN KEY (newsletter_id) REFERENCES newsletters(id) ON DELETE
CASCADE,
  ADD CONSTRAINT fk_pne_tenant      FOREIGN KEY (tenant_id)      REFERENCES tenants(id)      ON DELETE
CASCADE;
```

RISK / ROLLOUT

Adding an FK validates existing rows under lock. With no production users, run as a normal migration after checking for orphans (`SELECT ... LEFT JOIN ... WHERE parent.id IS NULL`). In production the safe pattern is `ADD CONSTRAINT ... NOT VALID`; then `ALTER TABLE ... VALIDATE CONSTRAINT ...`; (validation takes only a `SHARE UPDATE EXCLUSIVE` lock). Decide CASCADE vs NO ACTION per table deliberately — CASCADE on `emails` means tenant deletion silently removes mail; that is presumably the point, but write it down.

P2 D-4 `ms_oauth_tokens.tenant_id` / `google_oauth_tokens.tenant_id` are text

Every other table stores `tenant_id` as `bigint`; these two store it as text (the service layer writes `String(tenantId)`). Consequences: no FK to `tenants` is possible, joins need casts, the composite tenant-safety pattern can't apply, and a malformed value (`'42 '`, `'null'`) would simply sit there. These are the two most sensitive tables in the database (live mailbox tokens) — they deserve the strongest, not the weakest, integrity.

FIX

```
-- Verify: every value must be digits only
SELECT tenant_id FROM ms_oauth_tokens WHERE tenant_id !~ '^[0-9]+$'
UNION ALL SELECT tenant_id FROM google_oauth_tokens WHERE tenant_id !~ '^[0-9]+$';

ALTER TABLE ms_oauth_tokens      ALTER COLUMN tenant_id TYPE bigint USING tenant_id::bigint;
ALTER TABLE google_oauth_tokens  ALTER COLUMN tenant_id TYPE bigint USING tenant_id::bigint;
ALTER TABLE ms_oauth_tokens      ADD CONSTRAINT fk_ms_oauth_tenant      FOREIGN KEY (tenant_id) REFERENCES
tenants(id) ON DELETE CASCADE;
ALTER TABLE google_oauth_tokens  ADD CONSTRAINT fk_google_oauth_tenant  FOREIGN KEY (tenant_id) REFERENCES
tenants(id) ON DELETE CASCADE;
```

RISK

Table rewrite (instant at 1 row; brief even at thousands). Requires a matching TypeScript change: `kysely.models.ts` and both OAuth services must switch from string to number tenant IDs — small, mechanical, and type-checked end to end.

P2 D-5 `timestamp without time zone` in nine tables — including mixed usage inside one table

The email domain (`emails.deleted_at`, `email_bodies`, `email_headers` incl. `date_sent`, `email_recipients`, `email_attachments`, `email_drafts`, `email_trash`), `donations`, and `files` use tz-less timestamps, while ~50 other tables — and even `emails.created_at` in the same table — use `timestamptz`. A tz-less timestamp stores the wall-clock of whatever `TimeZone` the writing session had; the moment an app server, worker, or DBA session runs in a different zone, stored instants silently shift. `email_headers.date_sent` (real-world email timestamps) and `donations.created_at` (campaign-finance reporting) are exactly the columns where that matters.

FIX

```
-- Safe now (dev wrote everything in one zone). Repeat per column:
SET timezone = 'America/New_York'; -- the zone dev data was written in – confirm before running
ALTER TABLE donations
  ALTER COLUMN created_at TYPE timestamptz USING created_at AT TIME ZONE 'America/New_York',
  ALTER COLUMN updated_at TYPE timestamptz USING updated_at AT TIME ZONE 'America/New_York';
-- ...and likewise for emails.deleted_at, email_bodies, email_headers (incl. date_sent),
-- email_recipients, email_attachments, email_drafts, email_trash, files.
```

RISK

Each ALTER rewrites the table — trivial today, a maintenance-window task once large. Choosing the wrong `AT TIME ZONE` shifts historic data; with no production users, the cheapest correct answer is to convert now and adopt “timestamptz always” as a lint-level convention (the Kysely models make the column type explicit).

P2 D-6 `sessions.user_id` has a sequence default

`sessions.user_id` — a foreign key to `authusers` — carries `DEFAULT nextval('sessions_user_id_seq')`, almost certainly a column created with `serial` by accident. If any code path ever inserts a session without an explicit `user_id`, Postgres will mint a fresh number and either violate the FK (best case) or attach the session to an arbitrary future user (worst case, once IDs collide).

FIX

```
ALTER TABLE sessions ALTER COLUMN user_id DROP DEFAULT;
DROP SEQUENCE sessions_user_id_seq;
```

RISK

None — the application always supplies `user_id` explicitly (verified in `sessions.repo.ts`).

P2 D-7 Status/enum columns without CHECK constraints

The codebase's own best practice is text + CHECK (used on `background_jobs.status`, `tasks.status`, `web_forms.status`, etc.). These columns hold enumerated values but accept anything: `emails.status`, `donations.status`, `data_imports.status`, `data_exports.status`, `webhook_events.status`, `workflows.status`, `workflow_enrollments.status`, `task_subtasks.status`, `tasks.priority`, `notifications.type`, `persons.opt_in_status`, `persons.preferred_contact`, `households.geocoding_status`, `web_forms.form_type`. One typo'd writer (or one Zapier payload) creates a status no `@switch` in the frontend handles.

FIX (ONE MIGRATION; REPRESENTATIVE STATEMENTS)

```
ALTER TABLE donations      ADD CONSTRAINT chk_donations_status CHECK (status IN
('pending','completed','failed','refunded'));
ALTER TABLE emails         ADD CONSTRAINT chk_emails_status     CHECK (status IS NULL OR status IN
('open','closed','pending'));
ALTER TABLE workflows      ADD CONSTRAINT chk_workflows_status CHECK (status IN
('draft','active','paused','archived'));
-- ...one per column above; derive each value list from the Zod schema in libs/common (they are the source
of truth).
```

RISK

Take the allowed values from the corresponding Zod schemas so DB and API can't drift; validate existing rows first (`SELECT DISTINCT status FROM ...`). Adding a CHECK scans the table once. Also: the orphaned enum type `recipient_kind` (created but never used — `email_recipients.kind` is text+CHECK) should be dropped: `DROP TYPE recipient_kind;`

P3 D-8 NOT NULL and default gaps on auth-critical columns

- `sessions.expires_at` is nullable — a NULL session never expires. The revocation work on the current branch makes sessions the source of truth; give it `NOT NULL` (backfill first) or a CHECK documenting that NULL means non-expiring by design.
- `sessions.status` nullable with default `'active'` — make NOT NULL.
- `authusers.role` nullable (CHECK explicitly allows NULL) and `authusers.tenant_id` nullable — if NULL role means “pending invite,” encode that as a real state (`'invited'`) instead; NULL branches in authorization code are where privilege bugs live.
- `web_forms.slug` nullable while published forms are addressed by slug: `ADD CONSTRAINT chk_web_forms_published_slug CHECK (status <> 'published' OR slug IS NOT NULL)`.
- Cosmetic: `lists.is_dynamic`, `tags.deletable`, `map_peoples_tags.deletable`, `emails.status`, `google/ms_oauth_tokens.created_at/updated_at` are nullable with defaults — booleans with defaults should be NOT NULL.

RISK

Each `SET NOT NULL` scans the table; backfill NULLs first. The `authusers.role` change touches authorization logic — coordinate with the session-revocation branch.

P3 D-9 Primary-key and identifier conventions drift across table generations

Three PK shapes coexist: tenant-composite (`id, tenant_id`) (persons, tasks, events...), reversed (`tenant_id, id`) (`form_submissions`), and plain `id` on tenant-scoped tables (`emails`, `newsletters`, `workflows`, `workflow_steps`, `workflow_enrollments`, `files`, `settings`, `webhook_events`, `potential_duplicates`). The composite pattern plus `UNIQUE(id)` is a good, deliberate design (it lets children FK either the plain `id` or the tenant-safe pair); the drift is that newer non-composite tables can never participate in composite tenant-safe FKs. Similarly, 48 tables use `serial`-style sequences while the two newest use identity columns; and constraint/index names mix `_pk/_pkey`, `_key/_index/_idx/_unique`, including one typo (`fk_househods_tenant_id`) and misleading names (`households_tag_map_tenant_person_tag_index` on the households map; `idx_user_activity_user` which actually covers (`tenant_id, user_id`)).

RECOMMENDATION

Don't rewrite history — codify the target convention in `pplcrm-add-entity` (composite PK (`id, tenant_id`) + `UNIQUE(id)`, identity columns, `timestampz`, CHECK enums, `fk_<table>_<col>` naming) and apply it to new tables. Fix the typo when the households FK is next touched: `ALTER TABLE households RENAME CONSTRAINT fk_househods_tenant_id TO fk_households_tenant_id;`

P3 D-10 Duplicated data inside the email domain

`emails.body` coexists with `email_bodies.body_html` (a 1:1 table created to keep the big blob out of list scans — good), and `emails.to_email` coexists with the normalized `email_recipients` rows. Two sources of truth invite drift: which one does the UI trust after an edit? Similarly `data_imports` stores both `tag_id` and `tag_name`.

RECOMMENDATION

Pick the normalized home for each fact and demote the copy to a documented, derived cache (or drop it): keep `email_bodies` as the body's home and drop `emails.body`; keep `emails.preview` (derived, fine); keep `email_recipients` as truth and treat `to_email` as a display cache. Verify no reader depends on the dropped column before the migration; the donor-snapshot columns on `donations` / `donation_pledges` are *not* in this category — snapshotting donor data at transaction time is correct for finance records.

P3 D-11 `updated_at` maintenance is trigger-based on 26 tables, app-based (or absent) on ~25 others

Tables with an `updated_at` column but no `set_updated_at` trigger include `donations`, `files`, `email_recipients`, `email_attachments`, `email_trash`, `data_imports`, `data_exports`, `events`, `event_ticket_types`, `event_registrations`, all six `map_*` tables, `webhook_events`, `workflow_steps/enrollments`, `user_activity`, and `zapier_subscriptions`. On those, `updated_at` is whatever the app remembered to set — several update paths don't.

FIX

```
-- One migration, one statement per table:
CREATE TRIGGER trg_donations_updated_at BEFORE UPDATE ON donations
FOR EACH ROW EXECUTE FUNCTION set_updated_at(); -- ...repeat for the tables listed above
```

RISK

Negligible; one extra per-row trigger on updates. Skip append-only tables where `updated_at` is meaningless (`form_submissions` and `newsletter_events` already correctly omit the column). Also drop the orphaned function `sync_email_has_attachments()` — no trigger references it.

P3 D-12 JSONB usage is mostly sound; two patterns to watch

JSONB is used where shapes are genuinely dynamic (form `fields` / `answers`, job `payload`, list `definition`, Stripe payloads) — correct calls. Two soft spots: (1) six tables carry a column literally named `json` (persons, households, campaigns, companies, tenants, profiles) — an undocumented grab-bag with no schema; either name what lives there or remove it before data accretes. (2) `newsletters.target_lists` / `web_forms.target_lists` / `target_tags` hold arrays of IDs as JSONB with no referential integrity — deleting a list leaves dangling IDs inside documents. Acceptable as denormalized audience descriptions; just ensure readers treat missing IDs as soft-deleted (the UI already tolerates this).

4 Indexing findings and recommendations

245 indexes exist across 64 tables — a high count driven by three legitimate strategies (tenant-leading composites for grid queries, GIN trigram indexes for `ILIKE '%...%'` search, tsvector GINs for full-text) plus a fourth, illegitimate one: duplicates. The usage counters below are development-workload numbers and are quoted only where structure alone already decides the question.

P2 I-1 **Fifteen exactly-duplicate indexes — drop list**

Each pair indexes identical key columns on the same table. Every row write maintains both; the planner needs only one. In each case the survivor is the constraint-backed (or better-named) twin.

```
DROP INDEX authusers_email_index;           -- dup of UNIQUE authusers_email_key
DROP INDEX sessions_session_id_index;       -- dup of sessions_pkey
DROP INDEX sessions_refresh_token_index;    -- dup of UNIQUE sessions_refresh_token_key
DROP INDEX passkeys_credential_id_idx;      -- dup of UNIQUE passkeys_credential_id_key
DROP INDEX google_oauth_tokens_tenant_idx;  -- dup of UNIQUE
google_oauth_tokens_tenant_id_key
DROP INDEX ms_oauth_tokens_tenant_idx;      -- dup of UNIQUE ms_oauth_tokens_tenant_id_key
DROP INDEX idx_donations_stripe_session;    -- dup of UNIQUE donations_stripe_session_id_key
DROP INDEX idx_email_bodies_email_id;      -- dup of UNIQUE unique_email_bodies_email_id
DROP INDEX idx_email_headers_email_id;     -- dup of UNIQUE unique_email_headers_email_id
DROP INDEX households_tag_map_tenant_person_tag_index; -- dup of PK map_households_tags_pk
DROP INDEX peoples_tag_map_tenant_person_tag_index; -- dup of PK map_peoples_tags_pk
DROP INDEX idx_map_lists_households;       -- dup of PK map_lists_households_pk
DROP INDEX idx_map_lists_persons;          -- dup of PK map_lists_persons_pk
DROP INDEX idx_user_activity_user;         -- identical to idx_user_activity_tenant_user
(tenant_id, user_id)
DROP INDEX idx_user_activity_entity_id;     -- identical to idx_user_activity_tenant_entity
(tenant_id, entity, entity_id)
```

RISK

None to query plans — an identical index remains for every drop. In production, use `DROP INDEX CONCURRENTLY`.

Where dev stats show scans on the doomed twin (e.g. `authusers_email_index`, 1,898 scans), the planner will simply switch to the surviving identical index.

P2 I-2 Sixteen prefix-redundant indexes — a leading subset of a wider index

A B-tree on (a) is almost always redundant next to one on (a, b) — the wider index serves WHERE a = ... scans too. Candidates, with the wider index that covers them:

DROP CANDIDATE	COVERED BY
idx_companies_tenant (tenant_id)	idx_companies_tenant_email / _industry
idx_donations_tenant (tenant_id)	idx_donations_person (tenant_id, person_id)
idx_teams_tenant (tenant_id)	idx_teams_tenant_captain
tasks_tenant_index (tenant_id)	idx_tasks_tenant_status / _due / _assigned
idx_volunteer_events_tenant, idx_volunteer_events_tenant_start	idx_volunteer_events_dates (tenant_id, start_time, end_time)
idx_volunteer_shifts_tenant (tenant_id)	idx_volunteer_shifts_event / _person
idx_email_recipients_email_id (email_id)	idx_email_recipients_kind (email_id, kind, pos)
idx_email_read_states_user (tenant_id, user_id)	PK (tenant_id, user_id, email_id)
campaigns_map_tenant_campaign_index	PK (tenant_id, campaign_id, user_id)
idx_map_teams_lists_team, idx_map_teams_persons_team	their PKs (tenant_id, team_id, ...)
zapier_subscriptions_tenant_id_idx	UNIQUE (tenant_id, event_type)
idx_newsletter_events_tenant_newsletter	idx_newsletter_events_type (tenant_id, newsletter_id, event_type)
idx_notifications_tenant_user	idx_notifications_read (tenant_id, user_id, read)
idx_emails_tenant_folder <i>or</i> idx_emails_tenant_active	same columns; the partial (WHERE deleted_at IS NULL) duplicates the full one — keep the full, drop the partial

RISK

Low, with one honest caveat: a single-column index is slightly smaller and can be marginally faster for pure tenant_id scans. At this schema's cardinalities the difference is noise; the write savings across 16 indexes are not. Keep campaigns_tenant_index, newsletters_tenant_idx, web_forms_tenant_index, idx_workflow_enrollments_tenant_id, idx_files_tenant, idx_workflows_tenant_id — they are each table's only tenant-leading index.

P2 I-3 Unindexed foreign keys that will actually hurt: RI-trigger paths and hot lookups

100 FK columns lack a supporting index. Most are `createdby_id` / `updatedby_id` audit columns — deliberately **not** worth indexing (they're never queried, and 60 extra indexes would cost more than they save; the scheduled user-deletion job can afford its rare scans). The ones that matter are FKs on *delete paths and real query paths*. Note the subtlety: the map tables' tenant-leading PKs do *not* help referential-integrity triggers, because `DELETE FROM tags WHERE id=...` fires `... WHERE tag_id = $1` with no tenant qualifier — a sequential scan of the map table per deleted parent row.

FIX — CREATE THESE

```
-- Hot application path: "people in this household" + placeholder reassignment on delete
CREATE INDEX idx_persons_tenant_household ON persons (tenant_id, household_id);

-- RI cascade targets (single-column FKs with no leading index anywhere):
CREATE INDEX idx_map_peoples_tags_tag      ON map_peoples_tags (tag_id);      -- tag deletes: 17k+ in
dev already
CREATE INDEX idx_map_households_tags_tag   ON map_households_tags (tag_id);
CREATE INDEX idx_map_lists_persons_person ON map_lists_persons (person_id);  -- person deletes/merges
CREATE INDEX idx_map_lists_households_hh   ON map_lists_households (household_id);
CREATE INDEX idx_map_teams_persons_person  ON map_teams_persons (person_id);
CREATE INDEX idx_map_teams_lists_list      ON map_teams_lists (list_id);
CREATE INDEX idx_email_comments_email      ON email_comments (email_id);      -- currently NO index at
all
CREATE INDEX idx_workflow_enrollments_person ON workflow_enrollments (person_id);
CREATE INDEX idx_volunteer_shifts_event_ri ON volunteer_shifts (event_id);    -- CASCADE from
volunteer_events
CREATE INDEX idx_volunteer_shifts_person_ri ON volunteer_shifts (person_id);  -- CASCADE from persons
CREATE INDEX idx_email_trash_folder         ON email_trash (from_folder_id);  -- CASCADE from
email_folders

-- SET NULL paths, kept tiny with partial indexes:
CREATE INDEX idx_donations_pledge ON donations (pledge_id) WHERE pledge_id IS NOT NULL;
CREATE INDEX idx_event_registrations_ticket ON event_registrations (ticket_type_id) WHERE ticket_type_id
IS NOT NULL;
```

ALTERNATIVE WORTH CONSIDERING

Rather than adding bare-column indexes, convert the map tables' FKs to the composite pattern (`FOREIGN KEY (person_id, tenant_id) REFERENCES persons(id, tenant_id)`, as `event_registrations` and `person_connections` already do). Then the RI trigger query includes `tenant_id`, existing tenant-leading indexes serve it, and cross-tenant map rows become impossible. More work (drop/re-add ~12 FKs), strictly better result — scheduled as the strategic option in §7.

RISK

Each index adds write cost to its table; all of these are on delete/lookup paths that justify it. Use `CREATE INDEX CONCURRENTLY` in production (run outside the Kysely migration transaction); plain `CREATE INDEX` is fine today.

P2 I-4 **background_jobs : scheduler queries filter on payload->>'type' with no index**

Every scheduler tick runs ~10 dedupe queries of the form `WHERE status IN ('pending','processing') AND payload->>'type' = '...' (worker.ts)`. No index covers the JSONB expression, and the two partial/composite status indexes only narrow by status. Because completed jobs are never pruned (see P-3), the pending/processing subset stays small but the *table* grows without bound — and today's `idx_background_jobs_status_run_at` indexes all statuses including the dead majority.

FIX

```
CREATE INDEX idx_background_jobs_active_type
ON background_jobs ((payload->>'type'))
WHERE status IN ('pending','processing');
-- And make the claim-path index partial for the same reason (replaces the full one):
CREATE INDEX idx_background_jobs_claim ON background_jobs (queue, run_at)
WHERE status IN ('pending','processing');
DROP INDEX idx_background_jobs_status_run_at;
DROP INDEX idx_background_jobs_queue_status;
```

RISK

Verify the claim query's exact predicate before swapping the claim index (it must filter on the same status set for the partial index to apply). Pair with the retention job in P-3 — pruning is the real fix; these indexes make the interim cheap.

P3 I-5 **Missing index for session expiry sweeps**

Expiry is only checked per-request (`controller.ts:710`); nothing deletes expired rows in bulk, and if a sweep is added it would scan. Small table today; cheap to do right:

```
CREATE INDEX idx_sessions_expires_at ON sessions (expires_at) WHERE expires_at IS NOT NULL;
-- companion job (worker): DELETE FROM sessions WHERE expires_at < now() - interval '30 days';
```

P3 I-6 **Low-value and low-cardinality indexes**

- `idx_user_activity_activity` — indexes a low-cardinality text column *without* tenant scoping; no code path queries activity across tenants. Drop, or replace with `(tenant_id, activity)` if per-tenant activity filters exist.
- `idx_households_tenant_is_placeholder` — a boolean where at most one row per tenant is true. Replace with a partial index that also enforces the invariant: `CREATE UNIQUE INDEX idx_households_placeholder ON households (tenant_id) WHERE is_placeholder;`
- `idx_lists_tenant_is_dynamic`, `idx_households_tenant_geocoding`, `idx_emails_tenant_status` — two-valued second columns; harmless but near-useless next to a plain tenant index. Review when consolidating (I-2).
- The GIN trigram battery (18 indexes across 8 tables) is justified *only* for columns the grid actually offers “contains” search on. They are the most expensive indexes per write in the schema (persons: 6 GINs). Audit against the frontend column definitions and drop any not reachable from the UI.

GOOD I-7 What the index design gets right

- Unique constraints are properly index-backed throughout (partial unique indexes for nullable slugs — `idx_tenants_slug`, `idx_web_forms_tenant_slug`; expression unique for case-insensitive person emails — `lower(email)` with a trimmed-non-empty predicate).
- Dedup/idempotency keys exist where webhooks land: `webhook_events.stripe_event_id`, `newsletter_events.sg_event_id`, `donations.stripe_session_id` — all unique.
- Partial indexes are used idiomatically (`idx_data_exports_tenant_pending`, `idx_files_sha256 WHERE NOT NULL`, `potential_duplicates` partial uniques per entity kind).
- `form_submissions (tenant_id, form_id, created_at DESC)` matches its listing query exactly, ordering included.

5 Performance findings and recommendations

At 24 MB there are no performance *problems*, only performance *debts* — decisions that are free to change now and expensive later. Autovacuum is keeping up everywhere (dead tuples are single-digit percentages, all tables vacuumed/analyzed within the last day), and the heavy sequential-scan counters on small tables are the planner correctly preferring scans at tiny sizes, not missing indexes.

P1 P-1 PostgreSQL 14 reaches end-of-life on November 12, 2026

The server is 14.18 (Homebrew). In four months it stops receiving security fixes. Upgrading now — while the database is 24 MB, there are no users, and a dump/restore takes seconds — costs an afternoon; upgrading a production system later costs a project. PG 16/17 also bring direct wins for this schema: better parallelism, `pg_stat_io`, improved logical replication (useful for the RLS/reporting story), and PG15+ removes the PUBLIC `CREATE` grant on `public` (S-2) by default.

FIX

```
-- Homebrew path (dev): brew install postgresql@17
-- pg_dump pplcrm > pplcrm.sql (with the v17 pg_dump)
-- createdb pplcrm && psql pplcrm < pplcrm.sql
-- Re-run the app test suite against 17 before switching the default.
```

RISK

Low: the schema uses no deprecated features. Verify the two Homebrew services don't fight over port 5432, and re-check `pg_trgm` / `pgcrypto` extension versions after restore.

P2 P-2 Server configuration is stock dev defaults; no observability, no timeouts

Current settings: `shared_buffers=128MB`, `work_mem=4MB`, `effective_cache_size=4GB`, `statement_timeout=0`, `idle_in_transaction_session_timeout=0`, `log_min_duration_statement=-1`, and no `pg_stat_statements`. Fine for a laptop; three of these are dangerous defaults to inherit in production: no statement timeout means one runaway grid query holds a connection forever, no idle-in-transaction timeout means a hung app transaction blocks autovacuum and locks, and no query logging/statistics means the first performance incident starts blind.

FIX (PRODUCTION BASELINE; ROLE-LEVEL SO IT SHIPS WITH THE APP, NOT THE BOX)

```
-- Observability (needs restart + shared_preload_libraries='pg_stat_statements'):
CREATE EXTENSION pg_stat_statements;

-- Guard rails, scoped to the app role so migrations/admin sessions are unaffected:
ALTER ROLE pplcrm_app SET statement_timeout = '30s';
ALTER ROLE pplcrm_app SET idle_in_transaction_session_timeout = '60s';
ALTER SYSTEM SET log_min_duration_statement = '500ms';
ALTER SYSTEM SET random_page_cost = 1.1;           -- SSD storage
-- Memory sizing (adjust to the actual host; typical 4-8 GB instance):
ALTER SYSTEM SET shared_buffers = '1GB';           -- ~25% of RAM
ALTER SYSTEM SET effective_cache_size = '3GB';      -- ~75% of RAM
ALTER SYSTEM SET maintenance_work_mem = '256MB';
```

RISK

The 30s statement timeout must exceed the slowest legitimate query — large CSV exports run through the background worker (which can get its own `ALTER ROLE ... SET statement_timeout='10min'` if a separate worker role is created per S-2). Install `pg_stat_statements` in dev too, so the next review has real query data.

P2 P-3 Unbounded growth tables: no retention for jobs, webhook events, or activity

`background_jobs` is already the largest table in the database (2,152 rows, 1.2 MB) because completed and failed jobs are kept forever. The same applies to `webhook_events` (processed Stripe events), `user_activity`, `newsletter_events` (a prune job type exists in the worker — verify it is scheduled), and `sessions` (expired rows linger; deletion happens only on explicit sign-out paths). These are the tables that will actually be big in production; every one is append-mostly with an obvious time-based retention rule.

FIX — ONE RECURRING WORKER JOB (TRANSACTIONAL-OUTBOX STYLE, MATCHING EXISTING SCHEDULER PATTERNS)

```
DELETE FROM background_jobs WHERE status IN ('completed','failed') AND updated_at < now() - interval '30 days';
DELETE FROM webhook_events WHERE status = 'completed' AND processed_at < now() - interval '90 days';
DELETE FROM sessions WHERE expires_at IS NOT NULL AND expires_at < now() - interval '30 days';
-- user_activity / newsletter_events: define product retention (e.g. 12-24 months), delete in batches:
DELETE FROM user_activity WHERE id IN (
  SELECT id FROM user_activity WHERE created_at < now() - interval '24 months' LIMIT 10000);
```

RISK

Retention on `user_activity` and `newsletter_events` is a product/compliance decision (activity history is a feature; donation-adjacent logs may have legal retention floors) — set the window deliberately. Batch deletes (LIMIT + loop) to avoid long locks and huge WAL spikes; run in the worker's off-peak schedule.

P2 P-4 Connection pool is implicit: `new Pool(env.db)` with all defaults

`base.repo.ts:37` creates the pg pool with no `max` (defaults to 10), no `idleTimeoutMillis`, no `connectionTimeoutMillis`, and no `application_name`. The API server, the job worker (up to `WORKER_CONCURRENCY=4` concurrent claimers), the webhook worker, and LISTEN/NOTIFY listeners all share those 10 connections. Under a traffic burst plus a batch of jobs, requests will queue on the pool invisibly — with no timeout, they queue forever.

FIX (APPLICATION CHANGE, NOT SQL)

```
new Pool({
  ...env.db,
  max: env.dbPoolMax ?? 20,           // explicit; keep (api + worker) well under max_connections=100
  connectionTimeoutMillis: 5_000,     // fail fast instead of queueing forever
  idleTimeoutMillis: 30_000,
  application_name: 'pplcrm-api',      // and 'pplcrm-worker' in the worker → visible in
  pg_stat_activity
});
```

RISK

None today. When horizontal scaling arrives (multiple API instances), put PgBouncer (transaction mode) in front rather than raising per-instance `max` — but note transaction pooling interacts with the future RLS `SET LOCAL` pattern (S-1), which is designed for exactly that mode.

P3 P-5 Write amplification on the hottest tables

`persons` carries 17 indexes (six of them GIN), `households` 14 — index bytes outweigh heap bytes ~70:1 in dev. GIN trigram indexes are the expensive kind: every insert/update of an indexed text column fans out into many key entries. Imports are the workload that will feel this (10k persons inserted in dev testing already). This is a cost the search UX deliberately buys — the action is to stop paying it twice: the duplicate/redundant drops in I-1/I-2 remove five indexes from these two tables alone, and the trgm audit in I-6 may remove more. For bulk imports beyond ~100k rows, consider batching inserts and letting GIN `fastupdate` (already on by default) absorb churn; no schema change needed now.

P3 P-6 Partitioning: not needed now — here is the trigger point

No table justifies partitioning today, and premature partitioning would complicate every tenant-scoped query. The future candidates are the append-only event tables: `newsletter_events`, `user_activity`, `background_jobs`, `webhook_events`. Revisit when a table passes roughly 50–100 GB or ~100M rows *after* retention (P-3) is in place — retention usually removes the need entirely. If one does cross the line, use declarative range partitioning on `created_at` (monthly), which turns retention into instant `DROP PARTITION`. Do not partition by `tenant_id` — tenant counts are unbounded and hash partitioning would break the per-tenant index locality the current design already provides.

P3 P-7 Query-shape review: mostly clean, two patterns to watch

- **Grid pagination** (`getAllWithCounts`) issues a windowed page query plus a tenant-wide `count(*)` per grid load. Fine now; at large per-tenant row counts the count becomes the slow half — plan for an estimated count or cached totals when a tenant's persons exceed ~1M.
- **ILIKE '%...%' search** maps exactly onto the trgm GIN indexes — good. But grid filters also allow `notContains` / `notEquals` , which no index can serve; these will always scan the tenant's rows. Acceptable; just don't add UI defaults that trigger them.
- **1:1 splits** (`email_bodies` , `email_headers`) correctly keep large blobs out of list scans — a good N+1 *defense*, provided list queries join only when the detail view needs them (the repo code does this).
- **LISTEN/NOTIFY + SKIP LOCKED** workers avoid poll storms and lock contention by construction.

GOOD P-8 Vacuum & bloat: healthy

Autovacuum runs with default thresholds and is current on every table (last autovacuum/autoanalyze within a day on all active tables); dead-tuple counts are trivial (max 48 on `emails`). The high-churn tables to watch in production are `background_jobs` (15.7k updates in dev — every job claim/complete is an update) and `sessions` ; once retention is in place, default autovacuum settings will cope. If `background_jobs` update churn grows, a per-table tune is the first lever: `ALTER TABLE background_jobs SET (autovacuum_vacuum_scale_factor = 0.05);`

6 Security findings and recommendations

Application-layer security has been actively hardened (see `SECURITY-REVIEW.md` progress: revocable sessions, hashed tokens, rate limits, security headers). This section looks one layer down: what the *database* enforces if the application layer has a bug. The honest answer today is: very little — the database trusts the application completely.

P1 S-1 Tenant isolation has no database-level enforcement (no RLS)

Row-level security is not enabled on any table. Isolation rests on (a) the `local/no-unscooped-db-query` ESLint rule — which its own documentation says cannot catch queries built across multiple statements — and (b) the composite-FK pattern on newer tables. One missed `.where('tenant_id', ...)` that slips past the rule (a raw `sql` `` fragment, a dynamically built query) is a cross-tenant data leak of voter/donor PII.

RECOMMENDATION — RLS AS A SECOND WALL, PHASED

```
-- 1. App sets the tenant per transaction (works with PgBouncer transaction mode):
--   In the request context, inside each transaction:
--       SET LOCAL app.tenant_id = '42';
-- 2. Enable per table (start with persons/households/donations, expand outward):
ALTER TABLE persons ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON persons
  USING (tenant_id = current_setting('app.tenant_id')::bigint);
-- 3. The app role must not be the table owner (S-2), or RLS silently does not apply
--   (owners bypass their own policies unless FORCE ROW LEVEL SECURITY is set).
```

RISKS / TRADEOFFS

Real work: every request path must run inside a transaction that sets the GUC (the tRPC context is the natural place); the worker must set it per job; cross-tenant admin paths (the public `/f/:slug` form resolver, tenant provisioning, the three `ignoreTables`) need explicit policies or a dedicated role. Planner overhead is a constant-folded comparison per row — negligible against B-tree tenant indexes. Do it before launch; retrofitting RLS under live traffic is far harder. Interim mitigation (cheap, do now): extend the composite-FK pattern to the map tables (I-3) and add the missing tenant FKs (D-3).

P1 S-4 OAuth access & refresh tokens stored in plaintext

`ms_oauth_tokens` and `google_oauth_tokens` hold live Microsoft Graph / Google API access and refresh tokens as plain text (verified in `ms-oauth.service.ts` / `google-oauth.service.ts` — values are written as received). A database dump, a leaked backup, or any SQL-read primitive yields standing access to every connected tenant's mailbox. Everything else secret in this schema is hashed; these can't be hashed (they must be replayed), so they must be encrypted.

RECOMMENDATION

Encrypt at the application layer with AES-256-GCM using a key from the environment/KMS — *not* `pgp_sym_encrypt`, which would put the key into SQL text, logs, and `pg_stat_statements`. Pattern: `enc:v1:<iv>:<ciphertext>:<tag>` in the existing text columns, a small `crypto.ts` helper mirroring `token-hash.ts`, decrypt in the two OAuth services only. Migration: read-plaintext-or-encrypted during rollout, then a one-shot re-encrypt job. Rotate any tokens that predate encryption. Risk: a lost key bricks the connections — users just re-consent; acceptable.

P2 S-2 The application role owns every table; PUBLIC can CREATE in schema public

The app connects as `zeehamid`, which owns all 64 tables — so the runtime credential can `DROP TABLE`, `TRUNCATE`, `ALTER`, disable triggers, and (critically for S-1) bypass any future RLS by default. Separately, on PG14 the `public` schema still grants `CREATE` to `PUBLIC` (`{zee=UC/zee,=UC/zee}`), letting any connected role create objects and lay function-shadowing traps.

FIX (PRODUCTION ROLE LAYOUT)

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;           -- default in PG15+, do it now on 14

CREATE ROLE pplcrm_owner NOLOGIN;                      -- owns objects; migrations run as this
CREATE ROLE pplcrm_app LOGIN PASSWORD '...';           -- runtime; least privilege
GRANT USAGE ON SCHEMA public TO pplcrm_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO pplcrm_app;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO pplcrm_app;
ALTER DEFAULT PRIVILEGES FOR ROLE pplcrm_owner IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO pplcrm_app;
-- Reassign existing objects when cutting over: REASSIGN OWNED BY zeehamid TO pplcrm_owner;
```

RISK

The Kysely migrator runs at app boot today — it needs the owner credential, so split “migrate” (deploy step, owner role) from “serve” (app role). That's an app bootstrap change, best made before RLS depends on it.

P2 S-3 Local instance: trust authentication, SSL off — a production checklist, not a dev alarm

`pg_hba.conf` trusts all local and loopback connections (any local process can connect as the superuser with no password), and `ssl=off`. On a single-user dev laptop this is normal. The production checklist this implies:

- `scram-sha-256` auth for all rows in `pg_hba.conf` (password_encryption is already scram — good); no `trust`, no `md5`.
- TLS required end-to-end: server `ssl=on` with real certs; the app already supports `DB_SSL=true` (`env.ts`) — set it and prefer `sslmode=verify-full` semantics in the pg config.
- Network: database reachable only from app subnets; no public listener.
- Distinct credentials per environment; the dev password currently in `.env*` files must never be reused (and those files hold real-looking Google/Microsoft OAuth client secrets — gitignored, verified, but rotate them if they ever were committed anywhere, and move production secrets to a secret manager).

P2 S-5 Sensitive-data inventory: what lives where, and the two gaps

CATEGORY	COLUMNS	PROTECTION TODAY	VERDICT
Passwords	authusers.password	argon2id (64MB, t=3, p=4)	Good
Session/refresh tokens	sessions.session_id/refresh_token	SHA-256 digests (current branch)	Good
Reset & 2FA codes	authusers.password_reset_code, two_factor_code	SHA-256 + expiry + attempt cap	Good
Integration secrets	Zapier key, donation webhook token (in settings)	Hashed, show-once (recent commits)	Good
OAuth tokens	ms/google_oauth_tokens.access_token/refresh_token	Plaintext	S-4
WebAuthn	passkeys.public_key/credential_id	Public material — plaintext is correct	OK
Voter/donor PII	persons (contact, socials, opt-in), households (address + lat/long + district/precinct), donations/donation_pledges (donor identity + address)	Tenant scoping (app layer); TLS/at-rest = deployment concern	Depends on S-1/S-3
Behavioral PII	newsletter_events.ip/user_agent, sessions.ip_address/user_agent	No retention limit	P-3 retention

Gap 1 is S-4. Gap 2 is retention: IP-tagged behavioral rows accumulate indefinitely — set a retention window (P-3) and document it; for a product handling political affiliation data, deletion-on-request (tenant offboarding already schedules deletion — good) plus bounded logs is the defensible posture. Full-disk/backup encryption at the deployment layer is assumed and required.

P3 S-6 Audit trail is application-level and mutable

`user_activity` is a feature (activity feeds), not an audit log: the app role can UPDATE/DELETE it, nothing records role changes or permission grants at the DB level, and 26 tables get `updated_at` via trigger while the rest rely on the app (D-11). `authusers.previous_email/previous_role` is a nice lightweight history of the two most sensitive mutations. For production: enable `pgaudit` (log DDL + role/permission changes at minimum: `pgaudit.log = 'ddl, role'`), ship logs off-box, and treat `user_activity` as UX, not evidence. A trigger-based immutable audit table for `authusers` role/email changes is a cheap middle step if `pgaudit` is not available on the target platform.

P3 S-7 Backups & DR: out of scope here, in scope before launch

Not assessed on this dev instance (nothing to assess — no archiving configured). Before production: automated base backups + WAL archiving (pgBackRest/WAL-G or the managed provider's PITR), encrypted at rest, restore-tested on a schedule, with the OAuth-token encryption key (S-4) stored *separately* from database backups so a leaked backup remains inert.

GOOD**S-8 SQL injection posture: parameterized throughout, one guarded exception**

All data access goes through Kysely's builder (parameterized). The single `sql.raw(column)` interpolation in `base.repo.ts (buildRuleExpression)` receives only values from a server-side `columnMapping` allow-list — the client's `field` string is used solely as a lookup key and unmapped fields are dropped (verified). Filter *values* are always bound parameters. Keep that invariant explicit: a comment on `buildRuleExpression` stating “`column` must never contain client input” is worth adding, since the safety currently lives one call-frame away from the raw interpolation.

7 Prioritized action plan

Sequenced for a codebase with no production users: correctness first, then the structural work that is cheapest to do before data and traffic arrive. Effort is engineering time including tests and the `schema.sql` baseline refresh; “migration” means a normal timestamped Kysely migration per repo convention.

Wave 1 — quick wins (one sitting; all low risk)

REF	ACTION	IMPACT	RISK	EFFORT
D-1	Drop 11 duplicate FK constraints (one migration)	High (integrity clarity, write cost)	Low	~1 h
I-1	Drop 15 duplicate indexes (same migration)	Medium (write perf, cache)	Very low	~30 min
D-2	profiles: promote PK, dedupe auth_id FK	Medium	Low	~30 min
D-6	Drop sessions.user_id sequence default	Medium (latent bug)	None	10 min
I-3	Create the 14 RI/hot-path indexes	High at scale	Low	~1 h
I-4/P-3	background_jobs: partial type index + retention job (+ webhook/session sweeps)	High at scale	Low	~half day
S-2a	REVOKE CREATE ON SCHEMA public FROM PUBLIC	Medium	None	5 min
D-7/D-11	CHECK constraints from Zod enums; updated_at triggers; drop orphaned recipient_kind type & sync_email_has_attachments()	Medium	Low (validate rows first)	~half day

Wave 2 — structural fixes while the database is small

REF	ACTION	IMPACT	RISK	EFFORT
D-3	Add missing FKs (email domain, auth tables, form_submissions, engagements) after orphan checks	High (integrity, tenant deletion)	Medium — deliberate CASCADE decisions	~1 day
D-4	OAuth tenant_id text→bigint + FKs (+ TS model changes)	Medium	Low	~half day
D-5	Convert 9 tables to timestamptz	High (latent correctness)	Medium — pick source TZ correctly	~half day
I-2/I-6	Consolidate prefix-redundant + low-value indexes; trgm audit vs. UI search columns	Medium	Low	~half day

P-1	Upgrade PostgreSQL 14 → 17 (dev now, pin for prod)	High (EOL Nov 2026)	Low	~half day
P-4	Explicit pool config + application_name; split worker role	Medium	Low	~2 h
D-8	NOT NULL tightening (sessions, authusers.role semantics)	Medium	Medium — auth logic	~half day

Wave 3 — pre-launch security architecture

REF	ACTION	IMPACT	RISK	EFFORT
S-4	Encrypt OAuth tokens at rest (app-layer AES-GCM, env/KMS key)	High	Medium — key management	~1–2 days
S-2	Owner/app role split; migrate-on-deploy instead of migrate-on-boot	High (enables RLS)	Medium	~1–2 days
S-1	RLS rollout: GUC in tRPC context/worker, policies on PII tables first; composite-FK conversion of map tables as the structural complement	Very high (second wall for tenant isolation)	Medium-high — touches every request path; test with the real-DB test harness	~1–2 weeks, phased
S-3/S-6/S-7/P-2	Production platform checklist: scram+TLS, pgaudit, PITR backups + restore drills, pg_stat_statements, timeouts, memory sizing	High	Low — additive	Deploy-time

SEQUENCING NOTE

Waves 1–2 are ordinary migrations; land each through the repo's quality gate and refresh `apps/backend/src/app/_migrations/schema.sql` once at the end (per `pplcrm-migrations`). Wave 3's role split (S-2) is a prerequisite for RLS (S-1) — owners bypass their own policies. If only one Wave-3 item fits before launch, choose RLS: it is the control that turns every residual scoping bug from “breach” into “empty result set.”

— End of report. Raw inventories (columns, constraints, indexes, statistics) captured during this review are reproducible with the catalog queries cited throughout; no database state was modified.